EMBODIED AGENT ARCHITECTURE

Tracking Agent

技术架构报告

一个运行在 robot / Pi 侧的 chat-first embodied agent kernel。项目当前最核心的价值，在于把持续感知、事件驱动 turn、状态单源与 skill 插拔化收敛成清晰、可解释、可演进的系统骨架。

Chat-first | Perception 常驻 | Single Runner | Session Truth | Skill-Pluggable

3+1
推荐运行形态

1
Session Truth

Pi + Direct
决策路径

3
已验证 Skills

Inspection Scope
本报告基于仓库实态检查整理，聚焦系统设计、组件职责、状态与数据流、部署方式和扩展路径，不下钻到函数级实现。

Project Goal

项目目标与问题 framing

- **目标不是单点 tracking demo**
  仓库正在从 tracking-specific runtime 收敛为通用 embodied agent kernel，tracking 和 speech 是当前最明确的能力样板。
- **要解决的是系统问题**
  持续感知、会话状态、多轮对话、技能编排和展示层需要稳定协作，但不能演化成过度工程化的 runtime 框架。
- **关键约束**
  perception 负责提供世界状态，runner 负责单轮处理，viewer 负责状态投影，skills 负责具体 capability。

Architecture Thesis

系统最值得汇报的不是某个 tracking 算法细节，而是已经形成了“持续感知只做感知、单轮 runner 只做编排、状态只有一个真相源、skills 通过统一 surface 接入、viewer 只是展示层”的整体技术形态。

Design Principles

架构原则

- **Chat-first, not perception-first**
  turn 由聊天、脚本或 loop 事件触发；perception 只提供 grounded context。
- **Perception is the only always-on subsystem**
  只有 perception 常驻运行并持续写 observation 与 snapshot。
- **Single runner path**
  所有 turn 最终归并到 PiAgentRunner 的同一条主处理路径。
- **Single persisted state truth**
  session.json 是 agent-owned state 的主真相源。
- **Skills are ordinary modules**
  tracking、speech、web_search 都通过统一 skill surface 接入。
- **Viewer is a read-only projection**
  viewer 聚合并展示状态，但不反向驱动业务逻辑。

System Overview

系统全景与架构分层

三条运行平面

1. **Continuous Perception Plane**
   scripts/run_tracking_perception.py 持续读取 camera / video，经过 LocalPerceptionService 写入 perception snapshot 与 keyframes。

2. **Turn Orchestration Plane**
   backend/cli.py 与 PiAgentRunner 负责一次 turn 的上下文构造、skill 路由、Pi 调用或 deterministic direct path。

3. **Presentation Plane**
   agent_viewer_stream 聚合 session + observation + viewer modules，经 websocket 提供给 tracking-viewer。

高层结构关系

Trigger Layer
User / Script / Tracking Loop

Turn Orchestration
backend/cli.py -> PiAgentRunner -> Pi or direct skill

State Layer
active_session.json + session.json + perception snapshot.json

Capability Layer
skills/tracking + skills/speech + skills/web_search

Presentation Layer
viewer stream websocket -> React app

Continuous Perception
camera / video -> perception service -> snapshot

总体架构分层

Layer	代表对象	职责	架构意义
Trigger	chat / loop / scripts	产生 turn	把事件触发从内核逻辑中剥离
Orchestration	runner + pi_protocol	统一处理一次事件	避免多条并行决策主链
State	session + snapshot	落盘与共享状态	确保单一真相源和可调试性
Capability	skills/<name>/	实现具体能力	新增能力不污染 backend 主干
Presentation	viewer stream + app	聚合并展示状态	保持 UI 为只读投影层

Major Components

主要组件与职责

组件	代表路径	主要职责	架构意义
Perception Service	backend/perception/	维护 observation window、keyframe、snapshot、CLI 读取接口	持续感知层，但不做高层 orchestration
Agent Runner	backend/agent/runner.py	接收 turn、生成上下文、调用 Pi 或 direct path、应用 payload	系统唯一主处理链
Pi Protocol	backend/agent/pi_protocol.py	定义 reasoning plane 的输入输出边界	让 LLM 决策与本地状态管理解耦
Session Persistence	backend/persistence/	维护 session.json 与 active_session.json	保证单一状态真相源
Skill Surface	backend/skills.py	统一 discovery、route summary、turn context、viewer module 聚合	让 backend 保持通用
Tracking Skill	skills/tracking/	target init、continue tracking、memory rewrite、viewer module	当前最成熟的 embodied capability
Speech / Web Search	skills/speech/ + skills/web_search/	TTS 与外部信息样例能力	证明 plugability 不依赖 tracking 特例
Viewer Stream	backend/agent_viewer_stream.py	聚合 agent / observation / modules 并经 websocket 输出	将执行态转成展示态
Frontend App	apps/tracking-viewer/	展示目标框、memory、conversation history、状态标签	read-only projection layer
Runtime Scripts	scripts/	启动 perception、loop、viewer、frontend、stack	部署与进程编排层

Skill 接口为什么关键

- **build_route_summary**
  每个 skill 用自己的摘要参与 turn routing，而不是强迫 Pi 去阅读整份 session。
- **build_turn_context**
  为 skill 暴露专属上下文，缩小推理输入面。
- **process_direct_init / turn**
  为 fragile、确定性强的路径保留稳态入口。
- **schedule_rewrite / build_viewer_module**
  把慢操作和展示扩展下沉给 skill 自己。

Tracking Skill 的系统位置

- **它是 single-turn skill，不拥有 perception loop**
  skills/tracking/SKILL.md 明确要求 tracking skill 只处理当前 turn。
- **deterministic 入口比自由推理更重要**
  init / track 走固定入口脚本，降低 fragile workflow 被 LLM 发散的概率。
- **memory rewrite 脱离关键路径**
  先确认目标与结果，再异步更新 tracking memory，优先保证主交互响应。

Perception -> Planning -> Action

Embodied Agent Loop

标准 turn 处理路径

1. **Perception 持续写世界状态**
   camera / video 输入被采样并形成 observation、detections、keyframes 和 persisted snapshot。

2. **外部事件触发一次 turn**
   来源可以是用户 chat、tracking continuation loop 或脚本入口。

3. **Runner 聚合上下文**
   读取 session.json 与 perception snapshot，生成 route context、skill context 和 turn context。

4. **Planning / Routing**
   如果满足 deterministic 条件则走 direct path；否则由 Pi 在 enabled skills 中完成单轮路由。

5. **Action / Result**
   skill 返回统一 JSON payload，包含 session_result、skill_state_patch、robot_response 等结构化结果。

6. **Persistence + Projection**
   runner 应用状态更新；viewer stream 从持久化状态生成可视化投影；必要时异步执行 rewrite worker。

Perception / Planning / Action 的实际含义

- **Perception**
  以 LocalPerceptionService 为中心，关注 frame、detections、window、snapshot 和 keyframe 保存。
- **Planning**
  以 PiAgentRunner + Pi routing 为中心，决定该轮应该走哪个 skill、采用 Pi path 还是 direct path。
- **Action**
  当前更偏 capability result，例如 tracking 的 track / wait / ask，speech 的 TTS 输出，而不是完整机器人运动控制栈。

为什么这不是一个“大 while 循环”

- **Key Point 1**
  仓库把持续感知和事件驱动 turn 明确拆开。
- **Key Point 2**
  这样做的结果是，系统既保留 embodied 状态感知能力，又避免把所有职责揉成一个难以替换、难以验证的大 runtime。
- **Key Point 3**
  tracking loop 只是 capability-oriented trigger，不是系统总控中心；viewer 只是投影层，不承担执行逻辑。

Perception 常驻 | Turn 单次处理 | Viewer 只读 | Rewrite 异步 | Loop 只是触发器

Relationships

Runtime / Backend / App / Data Flow

当前推荐运行形态：3 个长期进程 + 1 个可选前端

Perception

持续读取 camera / video，运行 detection / tracking，向共享状态写 observation。

Tracking Loop

轮询 session 与 perception，在目标已绑定时触发 continuation turn。

Viewer Stream

把聚合后的 agent / perception / module 状态通过 websocket 推给前端。

Frontend

React + Vite 演示界面，负责目标框、状态、记忆和对话记录的可视化。

五种关系视角

视角	代表对象	关系描述	架构含义
Runtime	perception、loop、viewer、frontend	共享同一 state-root / session_id 的进程集合	职责清晰，可独立重启与组合
Backend	backend/	perception、runner、persistence、viewer aggregation 所在内核层	把主线保持在最小闭环里
Skills	skills/<name>/	实现具体 capability 与 helper，不改变 backend 核心形状	继续长能力而不是长框架
App	apps/tracking-viewer	通过 websocket 消费聚合状态	UI 与内核解耦
Data Flow	session + snapshot + artifacts	共享文件状态连接多个进程与视图	减少依赖，增强可调试性

关键状态对象

Active Session

active_session.json 指向当前活跃会话，是 perception、viewer、CLI 的共同索引。

Session Truth

sessions/<id>/session.json 保存 latest_result、history、recent_frames、skill_cache 等 agent-owned state。

Perception Snapshot

perception/sessions/<id>/snapshot.json 保存最新 observation、recent window、stream status 与 saved keyframes。

Turn Artifacts

.runtime/pi-agent/requests/... 保存 route context、skill context、prompt 等 turn 证据。

设计理由、Tradeoffs 与可扩展性

关键设计取舍

- **Chat-first vs perception-first**
  turn 由聊天或事件触发，perception 只提供 grounded context；适合 agent interaction，但未来高频闭环控制仍需更硬实时的 control plane。
- **Single session truth vs 多份 memory**
  session.json 让调试和 viewer 聚合都有统一落点，但当前更偏单机和单活跃 session 形态。
- **File-backed shared state vs event bus**
  本地共享文件简单、直观、易部署，但不以分布式吞吐和强一致消息语义为目标。
- **Direct path vs full LLM mediation**
  tracking 的 init / track 支持 deterministic 入口，让 fragile workflow 更稳，但 skill 需要承担更明确的契约维护。
- **Async rewrite vs inline completion**
  主 turn 响应更快，但 memory 更新变成 eventual consistency。
- **Generic viewer shell vs tracking-only UI**
  viewer 可继续承载更多 skill 模块，但公共 schema 必须保持克制，避免重新长成大框架。

扩展性判断

- **已经具备的扩展点**
  新增 skill 可以放在 skills/<name>/，backend 自动发现，并在 route summary、turn context、viewer module、direct path 等协作点接入。
- **当前刻意不做的事**
  不引入复杂 plugin lifecycle、不引入消息总线、不把 tracking loop 升级为总控中心，以避免再次回到过度工程化路径。
- **最重要的判断**
  仓库已经具备“继续长能力而不是继续长框架”的结构条件。

部署判断

- **进程职责清晰**
  perception、loop、viewer 可分别重启和定位问题，不会形成单点超级进程。
- **Viewer 可选**
  前端不再是主流程依赖，既适合 headless，也适合演示。
- **状态共享简单**
  共享同一 state-root 和 session 路径即可运行，贴合 robot / Pi 侧轻量部署。
- **能力替换容易**
  perception 源、skills 和展示层都可以演进，而不需要先改内核。

Outlook

边界、Future Work 与总结

当前边界与成熟度

- **最成熟的 embodied capability 仍然是 tracking**
  speech 与 web_search 已经证明 plugability，但 embodied 主场景仍围绕 tracking。
- **状态共享主要是本地文件级协作**
  这非常适合单机部署和调试，但不等于已经具备跨机分布式架构。
- **Action plane 仍偏结果型输出**
  当前重点是 grounded 决策与能力执行，而不是完整机器人控制栈。
- **Perception 仍聚焦 tracking 场景**
  它已经是独立感知层，但还不是通用多传感器融合平台。

Future Work

- **1. 更强的部署韧性**
  为 perception、loop、viewer 增加更明确的健康检查、超时与恢复策略。
- **2. 更通用的 capability surface**
  在保持 skill surface 克制的前提下，接入更多 embodied 能力。
- **3. 更强的 observability**
  补齐 turn latency、skill routing、rewrite worker、session lifecycle 等指标。
- **4. 多设备 / 远程部署能力**
  如果跨机部署成为刚需，再从本地文件共享迁移到更明确的 service / bus 架构。

汇报结论

tracking_agent 当前最值得强调的成果，不是某一个 tracking 算法细节，而是已经把 embodied agent 方案压缩成一个可解释、可部署、可扩展的技术骨架。

持续感知只做感知 | 单轮 runner 只做编排 | 状态只有一个真相源 | skills 通过统一 surface 接入 | viewer 只是展示层

这让项目从“为 tracking 服务的一组脚本”演进为“以 tracking 为样板能力的 embodied agent kernel”。对外汇报时，应把这种系统级收敛能力作为主叙事。